



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών (ΣΗΜΜΥ)

8ο εξάμηνο

Ψηφιακά Συστήματα VLSI

1η εργαστηριακή άσκηση

Δημήτρης Βατάλας

(Α.Μ.: 03121093)

Αριστοτέλης Γεώργιος Δρίτσας

(Α.Μ.: 03122127)

Αθήνα, Μάρτιος 2026

Contents

Θέμα A.2: Δυαδικός αποκωδικοποιητής 3 σε 8	2
Θέμα B.2: Καταχωρητής ολίσθησης των 4 bits με παράλληλη φόρτωση	8
Θέμα B.3: Μετρητής 3 bit με είσοδο ενεργοποίησης και κρατούμενο εξόδου	12
1. Up/Down μετρητής χωρίς όριο	12
2. Up μετρητής με όριο	15
Θέμα 1: Ημιαθροιστής σε περιγραφή dataflow	18
Θέμα 2: Πλήρης αθροιστής σε περιγραφή structural	20
Θέμα 3: Παράλληλος αθροιστής των 4 bit σε περιγραφή structural	23
Θέμα 3: BCD Full Adder σε περιγραφή structural	27
Θέμα 4: 4-BCD Parallel Adder σε περιγραφή structural	30

Θέμα Α.2: Δυαδικός αποκωδικοποιητής 3 σε 8

Ο κώδικας σε vhdl για την περιγραφή του αποκωδικοποιητή και της αρχιτεκτονικής του σε dataflow vhdl είναι ο εξής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

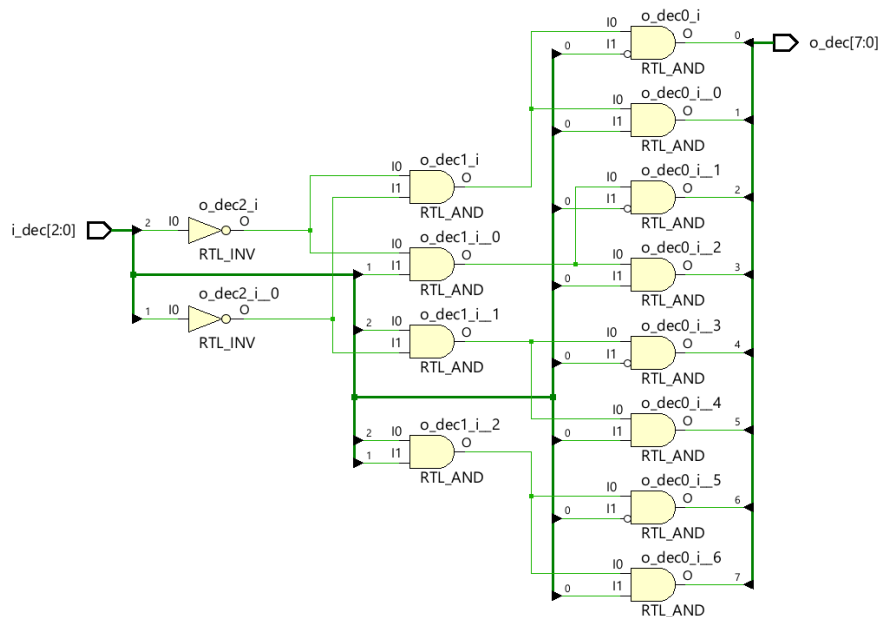
entity bin_dec is
    port
    (
        i_dec : in std_logic_vector(2 downto 0);
        o_dec : out std_logic_vector(7 downto 0)
    );
end bin_dec;

architecture Dataflow of bin_dec is

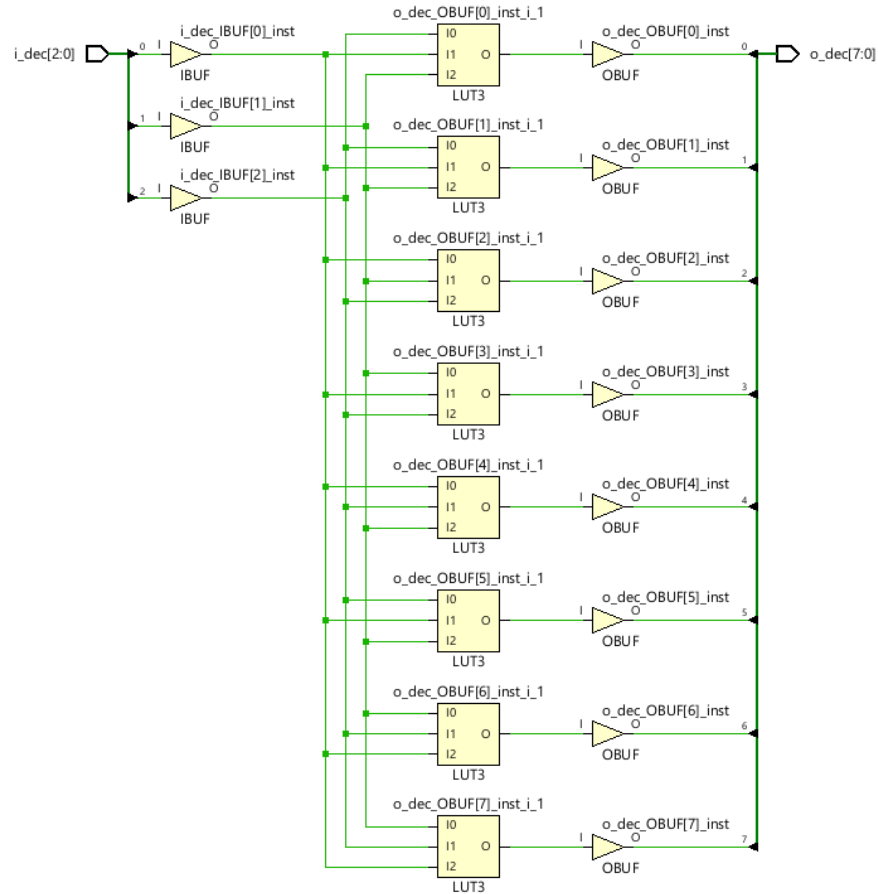
begin
    o_dec(0) <= (not i_dec(2)) and (not i_dec(1)) and (not i_dec(0));
    o_dec(1) <= (not i_dec(2)) and (not i_dec(1)) and i_dec(0);
    o_dec(2) <= (not i_dec(2)) and i_dec(1) and (not i_dec(0));
    o_dec(3) <= (not i_dec(2)) and i_dec(1) and i_dec(0);
    o_dec(4) <= i_dec(2) and (not i_dec(1)) and (not i_dec(0));
    o_dec(5) <= i_dec(2) and (not i_dec(1)) and i_dec(0);
    o_dec(6) <= i_dec(2) and i_dec(1) and (not i_dec(0));
    o_dec(7) <= i_dec(2) and i_dec(1) and i_dec(0);

end Dataflow;
```

Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Με τη χρήση του synthesis design παίρνουμε το εξής schematic:



Για να μπορούμε να ελέγξουμε την ορθή λειτουργία του αποκωδικοποιητή παράγουμε όλα τα πιθανά σήματα εισόδου και παρατηρούμε τις εξόδους σε καθένα από αυτά:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity bin_dec_tb is
-- Port ( );
end bin_dec_tb;

architecture tb of bin_dec_tb is
-- COMPONENT
component bin_dec is
port
(
i_dec : in std_logic_vector(2 downto 0);
o_dec : out std_logic_vector(7 downto 0)
);
end component;

-- SIGNALS
signal i_dec      : std_logic_vector(3-1 downto 0) := (others => '0');
```

```

signal o_dec      : std_logic_vector(8-1 downto 0) := (others => '0');

-- CONSTANTS
constant TIME_DELAY : time := 10 ns;

begin

    DUT : bin_dec
        port map (
            i_dec    => i_dec,
            o_dec    => o_dec
        );

    STIMULUS : process
    begin
        i_dec <= "000";
        wait for (1 * TIME_DELAY);

        i_dec <= "001";
        wait for (1 * TIME_DELAY);

        i_dec <= "010";
        wait for (1 * TIME_DELAY);

        i_dec <= "011";
        wait for (1 * TIME_DELAY);

        i_dec <= "100";
        wait for (1 * TIME_DELAY);

        i_dec <= "101";
        wait for (1 * TIME_DELAY);

        i_dec <= "110";
        wait for (1 * TIME_DELAY);

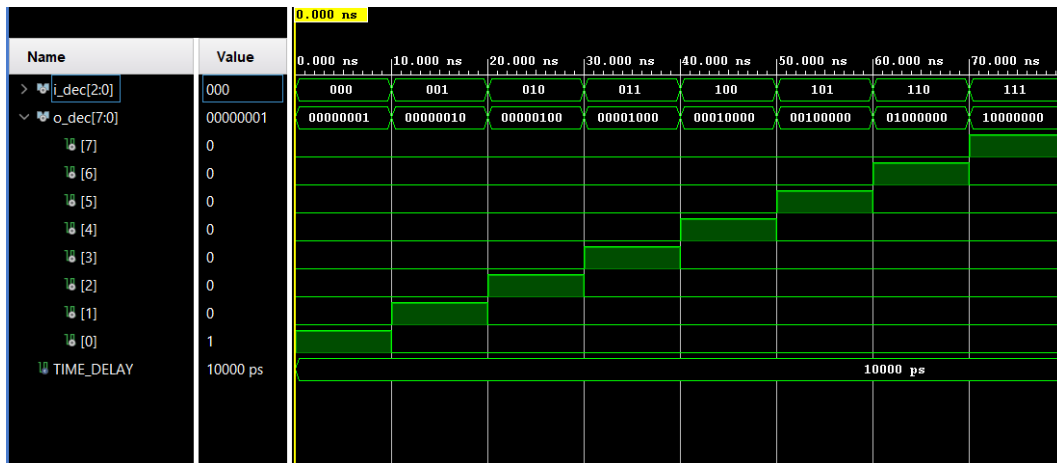
        i_dec <= "111";
        wait for (1 * TIME_DELAY);

        wait;

    end process;
end tb;

```

Τα αποτελέσματα της προσομοίωσης φαίνονται παρακάτω:



Παρατηρούμε την ορθή σχέση μεταξύ κάθε εισόδου και αντίστοιχης εξόδου.

Ο κώδικας σε vhdl για την περιγραφή του αποκωδικοποιητή και της αρχιτεκτονικής του σε behavioral vhdl είναι ο εξής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity bin_dec is
    port
    (
        i_dec : in std_logic_vector(2 downto 0);
        o_dec : out std_logic_vector(7 downto 0)
    );
end bin_dec;

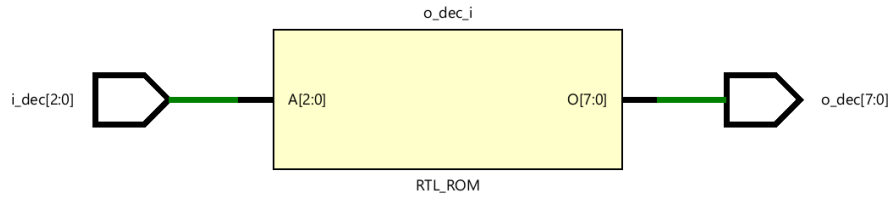
architecture Behavioral of bin_dec is

begin

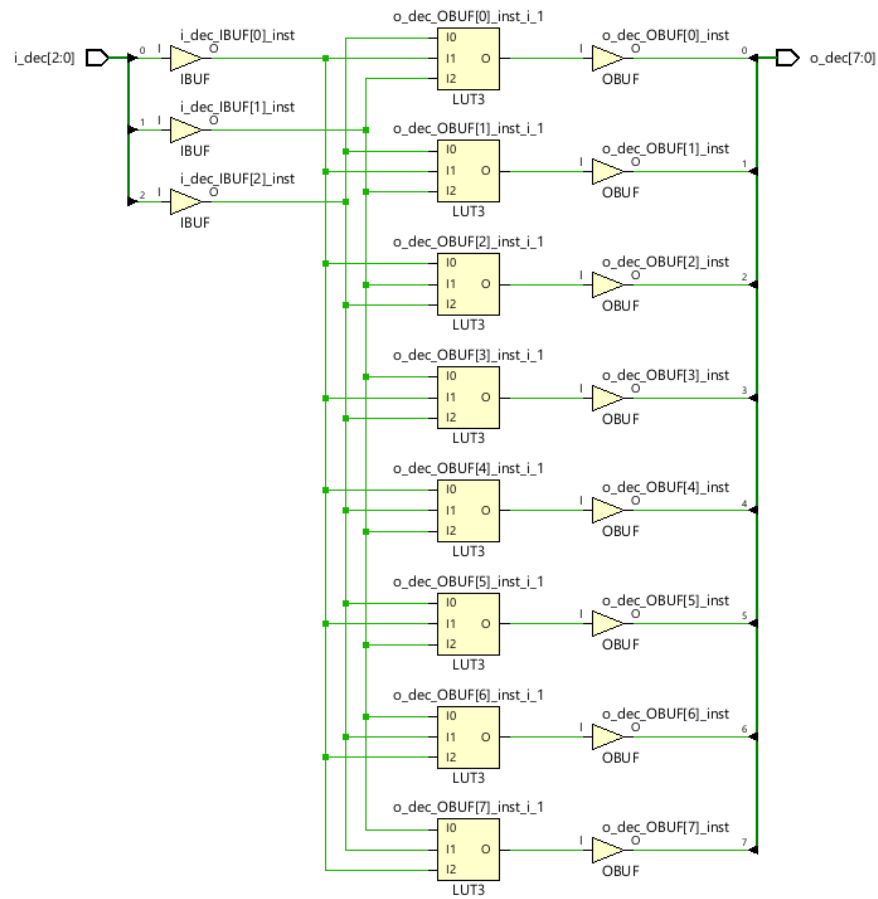
    ENC_MODULE: process(i_dec)
    begin
        case i_dec is
            when "000" => o_dec <= "00000001";
            when "001" => o_dec <= "00000010";
            when "010" => o_dec <= "00000100";
            when "011" => o_dec <= "00001000";
            when "100" => o_dec <= "00010000";
            when "101" => o_dec <= "00100000";
            when "110" => o_dec <= "01000000";
            when "111" => o_dec <= "10000000";
            when others => o_dec <= "00000000";
        end case;
    end process;

end Behavioral;
```

Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Με τη χρήση του synthesis design παίρνουμε το εξής schematic:



Μπορούμε να παρατηρήσουμε ότι για τις δύο διαφορετικές αρχιτεκτονικές, το schematic που προκύπτει από το στάδιο του elaborate design είναι διαφορετικό, καθώς εξαρτάται άμεσα από τον τρόπο με τον οποίο έχει περιγραφεί η αρχιτεκτονική στον κώδικα. Αντίθετα, το schematic που προκύπτει μετά το στάδιο του synthesis είναι το ίδιο και στις δύο περιπτώσεις. Αυτό δείχνει ότι, παρόλο που ο αποκωδικοποιητής έχει περιγραφεί με δύο διαφορετικές αρχιτεκτονικές σε επίπεδο κώδικα, μετά το synthesis προκύπτει η ίδια τελική λογική δομή και συνεπώς η ίδια υλοποίηση στο FPGA.

Για να μπορούμε να ελέγξουμε την ορθή λειτουργία του αποκωδικοποιητή παράγουμε όλα τα πιθανά σήματα εισόδου και παρατηρούμε τις εξόδους σε καθένα από αυτά:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```

use ieee.numeric_std.all;

entity bin_dec_tb is
-- Port ( );
end bin_dec_tb;

architecture tb of bin_dec_tb is
-- COMPONENT
component bin_dec is
port
(
i_dec : in std_logic_vector(2 downto 0);
o_dec : out std_logic_vector(7 downto 0)
);
end component;

-- SIGNALS
signal i_dec      : std_logic_vector(3-1 downto 0) := (others => '0');
signal o_dec      : std_logic_vector(8-1 downto 0) := (others => '0');

-- CONSTANTS
constant TIME_DELAY : time := 10 ns;

begin

DUT : bin_dec
port map (
i_dec      => i_dec,
o_dec      => o_dec
);

STIMULUS : process
begin
i_dec <= "000";
wait for (1 * TIME_DELAY);

i_dec <= "001";
wait for (1 * TIME_DELAY);

i_dec <= "010";
wait for (1 * TIME_DELAY);

i_dec <= "011";
wait for (1 * TIME_DELAY);

i_dec <= "100";
wait for (1 * TIME_DELAY);

i_dec <= "101";
wait for (1 * TIME_DELAY);

i_dec <= "110";
wait for (1 * TIME_DELAY);

i_dec <= "111";

```

```

        wait for (1 * TIME_DELAY);

    wait;

end process;
end tb;

```

Τα αποτελέσματα της προσομοίωσης φαίνονται παρακάτω:



Παρατηρούμε και σε αυτήν την περίπτωση την ορθή σχέση μεταξύ κάθε εισόδου και αντίστοιχης εξόδου.

Θέμα B.2: Καταχωρητής ολίσθησης των 4 bits με παράλληλη φόρτωση

Ο κώδικας σε vhdl για την υλοποίηση του καταχωρητή ολίσθησης, ο οποίος διαθέτει μία επιπλέον είσοδο (std_logic) για την επιλογή μεταξύ αριστερής και δεξιάς ολίσθησης, παρουσιάζεται παρακάτω:

Αξίζει να σημειωθεί ότι στον κώδικα έχει προστεθεί επίσης μία επιπλέον μεταβλητή, η ins_test, η οποία δεν είναι απαραίτητη για την ίδια τη λειτουργία του κυκλώματος. Η ύπαρξή της αποσκοπεί στη διευκόλυνση της επαλήθευσης κατά την προσομοίωση, καθώς επιτρέπει την άμεση παρατήρηση της τιμής που είναι αποθηκευμένη κάθε στιγμή στον καταχωρητή ολίσθησης.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shift_reg is
    Port (
        clk,rst,si,en,pl,ch_way: in std_logic;
        din: in std_logic_vector(3 downto 0);
        so: out std_logic;
        ins_test :out std_logic_vector(3 downto 0)
    );
end shift_reg;

architecture rtl of shift_reg is

    signal dff: std_logic_vector(3 downto 0);
    signal sf: std_logic;
begin
    edge: process (clk,rst)

```



```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity shift_reg_tb is
-- Port ( );
end shift_reg_tb;

architecture tb of shift_reg_tb is
-- COMPONENT
component shift_reg is
Port (
    clk,rst,si,en,pl,ch_way: in std_logic;
    din: in std_logic_vector(3 downto 0);
    so: out std_logic;
    ins_test :out std_logic_vector(3 downto 0)
);
end component;
-- SIGNALS
signal clk,rst,si,en,pl,ch_way : std_logic ;
signal din : std_logic_vector(3 downto 0);
signal so: std_logic ;
signal ins_test : std_logic_vector(3 downto 0) ;
-- CONSTANTS
constant CLOCK_PERIOD : time := 10 ns;

```

```
begin
```

```

    DUT : shift_reg
    port map (
        clk    => clk,
        rst    => rst,
        si     => si,
        en     => en,
        pl     => pl,
        ch_way => ch_way,
        din    => din,
        so     => so,
        ins_test =>ins_test
    );
    STIMULUS : process
    begin
        -----initialize signals-----
        rst    <= '0';    -- active low reset
        en     <= '0';
        pl     <= '0';
        ch_way <= '1';
        din    <= "1010";
        wait for CLOCK_PERIOD;

        -----load initial number-----
        rst <= '1';    -- release reset
        pl <= '1';
        wait for 4*CLOCK_PERIOD;

        -----test shift right-----
        pl <= '0';
        en <= '1';
        wait for 8*CLOCK_PERIOD;

        -----test reset-----

```

```

rst <= '0';
wait for 4*CLOCK_PERIOD;

-----load new number-----
en <= '0';
rst <= '1';
din <= "1001";
pl <= '1';
wait for 4*CLOCK_PERIOD;

-----test shift left-----
pl <= '0';
en <= '1';
ch_way <= '0';
wait for 8*CLOCK_PERIOD;

-----test enable-----
en <= '0';
wait for 4*CLOCK_PERIOD;

wait;
end process;

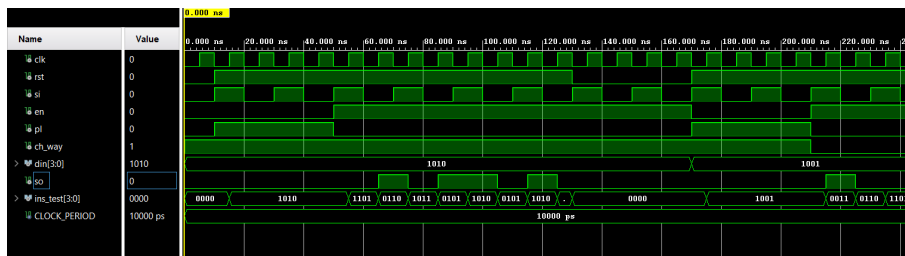
GEN_SI : process
begin
loop
si <= '0';
wait for CLOCK_PERIOD;
si <= '1';
wait for CLOCK_PERIOD;
end loop;
end process;

GEN_CLK : process
begin
clk <= '0';
wait for (CLOCK_PERIOD / 2);
clk <= '1';
wait for (CLOCK_PERIOD / 2);
end process;

end tb;

```

Για τον έλεγχο της ορθής λειτουργίας του καταχωρητή εξετάστηκαν οι βασικές καταστάσεις λειτουργίας του κυκλώματος, όπως reset, enable/disable, η παράλληλη φόρτωση δεδομένων και προφανώς η ολίσθηση των δεδομένων και προς τις δύο κατευθύνσεις, αφού στην προκειμένη περίπτωση μόνο λαμβάνοντας υπόψη όλες τις πιθανές εισόδους παράλληλης φόρτωσης που μπορούν να υπάρξουν η εξαντλητική επαλήθευση δεν ήταν εφικτή. Παρακάτω παρουσιάζονται ενδεικτικά αποτελέσματα της προσομοίωσης, για λόγους συντομίας δεν παρατίθενται όλες οι καταστάσεις που εξετάστηκαν, αλλά ένα αντιπροσωπευτικό τμήμα στο οποίο φαίνονται οι σημαντικότερες λειτουργίες του κυκλώματος και επιβεβαιώνεται η ορθή συμπεριφορά του.



Επομένως, μπορούμε να παρατηρήσουμε ότι οι τιμές τις εξόδου και η μορφή των σημάτων επαληθεύουν την ορθή λειτουργία του καταχωρητή που υλοποιήσαμε.

Θέμα Β.3: Μετρητής 3 bit με είσοδο ενεργοποίησης και κρατούμενο εξόδου

1. Up/Down μετρητής χωρίς όριο

Ο κώδικας σε vhdl για την περιγραφή του up/down μετρητή χωρίς όριο είναι ο εξής:

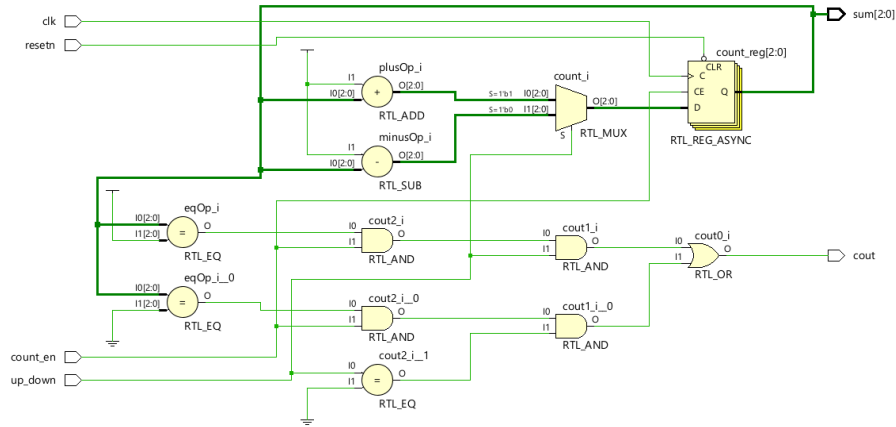
```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity counter_up_down is
    port(
        clk, resetn, count_en, up_down : in std_logic;
        sum : out std_logic_vector(2 downto 0);
        cout : out std_logic
    );
end counter_up_down;

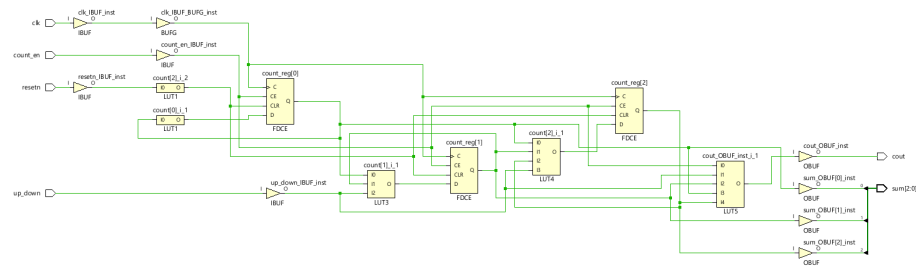
architecture rtl_nolimit of counter_up_down is

    signal count: std_logic_vector(2 downto 0);
begin
    process(clk, resetn)
    begin
        if resetn='0' then --reset is negative logic
            count <= (others=>'0');
        elsif clk'event and clk='1' then
            if count_en='1' then --enable counting positive logic
                case up_down is
                    when '1'=>
                        count<=count+1;
                    when '0' =>
                        count<=count-1;
                    when others =>
                        null;
                    end case;
                end if;
            end if;
        end process;
        sum <= count;
        cout <= '1' when (count = 7 and count_en = '1' and up_down = '1') or
            (count = 0 and count_en = '1' and up_down = '0')
            else '0';
    end rtl_nolimit;
```

Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Με τη χρήση του synthesis design παίρνουμε το εξής schematic:



Για την εκτέλεση της προσομοίωσης υλοποιήθηκε το παρακάτω testbench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity count_up_down_nolim_tb is
-- Port ( );
end count_up_down_nolim_tb;

architecture tb of count_up_down_nolim_tb is

-- COMPONENT
component counter_up_down is
port(
    clk, resetn, count_en, up_down : in std_logic;
    sum : out std_logic_vector(2 downto 0);
    cout : out std_logic
);
end component;

-- SIGNALS
signal clk, resetn, count_en, up_down : std_logic;
signal sum : std_logic_vector(2 downto 0);
signal cout : std_logic;

-- CONSTANTS
constant CLOCK_PERIOD : time := 10 ns;

begin
    DUT : counter_up_down
    port map(
```

```

        clk      => clk,
        resetn   => resetn,
        count_en => count_en,
        up_down  => up_down,
        sum      => sum,
        cout     => cout
    );

    STIMULUS : process
    begin

        up_down  <= '1';
        count_en<='0';
        resetn  <='0'; --initialize the counter to zero
        wait for (1*CLOCK_PERIOD);

        resetn  <='1';
        count_en<='1'; --enable counting
        wait for (14*CLOCK_PERIOD); --count two times from zero to 7

        resetn  <='0'; --test reset
        wait for (1*CLOCK_PERIOD);

        resetn  <='1';
        up_down  <='0'; --count down
        wait for (14*CLOCK_PERIOD);

        wait ;

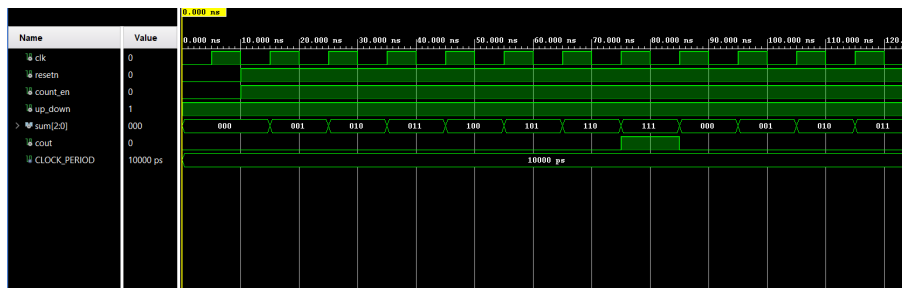
    end process;

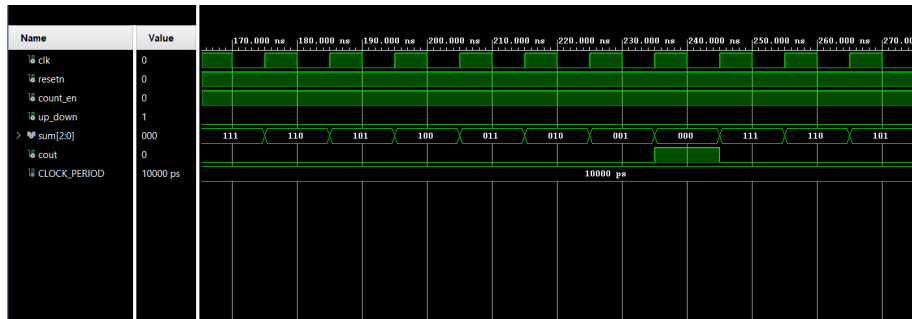
    GEN_CLK : process
    begin
        clk <= '0';
        wait for (CLOCK_PERIOD / 2);
        clk <= '1';
        wait for (CLOCK_PERIOD / 2);
    end process;
end tb;

```

Απο το συγκεκριμένο tb παρατίθενται δυο στιγμιότυπα για να επεικονιστή και η μέτρηση προς τα πάνω και η μέτρηση προς τα κάτω.

Όπως θα φανεί και στα στιγμιότυπα από τις προσομοιώσεις, έχουμε βάλει να εμφανίζεται «μήνυμα υπερχείλισης» θέτοντας το cout ίσο με 1, τόσο στην περίπτωση που φτάνουμε στο 7 και έχουμε μέτρηση προς τα πάνω, αλλά και στην περίπτωση που φτάνουμε στο 0 και έχουμε μέτρηση προς τα κάτω.





Απο τις πάραπάνω εικόνες τετριμένα προκύπτει η ορθή λειτουργία του counter μας τόσο για το Mode μέτρησης προς τα πάνω όσο και για το Mode μέτρησης προς τα κάτω.

2. Up μετρητής με όριο

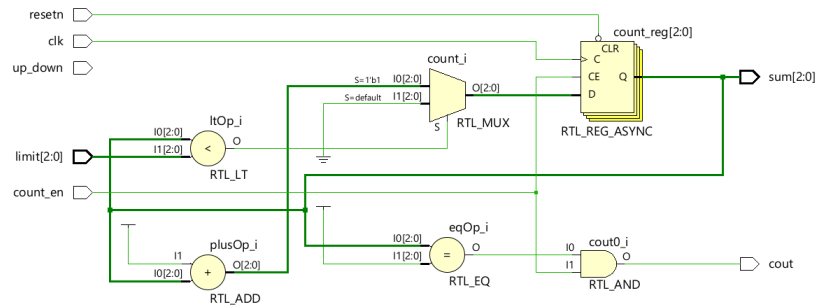
Ο κώδικας σε vhdl για την περιγραφή του up μετρητή με όριο είναι ο εξής:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

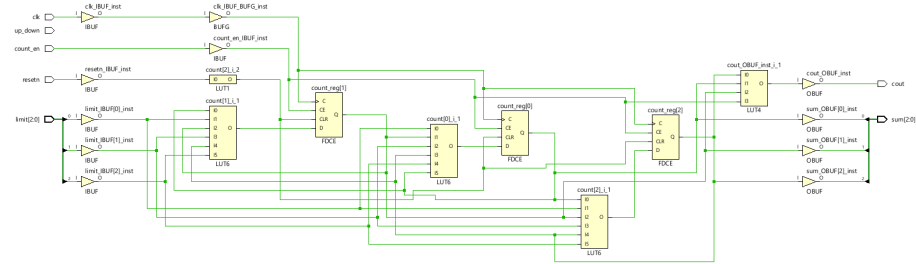
entity counter_up_down_lim is
    port(
        clk,resetn,count_en,up_down : in std_logic;
        limit : in std_logic_vector(2 downto 0);
        sum : out std_logic_vector(2 downto 0);
        cout : out std_logic
    );
end counter_up_down_lim;

architecture rtl_lim of counter_up_down_lim is
    signal count : std_logic_vector(2 downto 0);
    begin
        process(clk, resetn, limit)
        begin
            if resetn='0' then
                -- Ασύγχρονος μηδενισμός
                count <= (others=>'0');
            elsif clk'event and clk='1' then
                if count_en = '1' then
                    -- Μέτρηση μόνο αν count_en='1'
                    if count<limit then
                        count <= count+1;
                    else
                        -- Αλλιώς τον μηδενίζουμε
                        count<=(others=>'0');
                    end if;
                end if;
            end if;
        end process;
        sum<= count;
        cout <= '1' when count=7 and count_en='1' else '0';
    end rtl_lim;
```

Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Με τη χρήση του synthesis design παίρνουμε το εξής schematic:



Για την εκτέλεση της προσομοίωσης υλοποιήθηκε το παρακάτω testbench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity count_up_limit_tb is
-- Port ( );
end count_up_limit_tb;

architecture tb of count_up_limit_tb is

-- COMPONENT
component counter_up_down_lim is
port(
    clk, resetn, count_en, up_down : in std_logic;
    limit : in std_logic_vector(2 downto 0);
    sum : out std_logic_vector(2 downto 0);
    cout : out std_logic
);
end component;

-- SIGNALS
signal clk : std_logic := '0';
signal resetn : std_logic := '0';
signal count_en : std_logic := '0';
signal up_down : std_logic := '0';
signal limit : std_logic_vector(2 downto 0) := "000";
signal sum : std_logic_vector(2 downto 0);
signal cout : std_logic;
```

```

-- CONSTANTS
constant CLOCK_PERIOD : time := 10 ns;

begin
  DUT : counter_up_down_lim
  port map(
    clk      => clk,
    resetn   => resetn,
    count_en => count_en,
    up_down  => up_down,
    limit    => limit,
    sum      => sum,
    cout     => cout
  );

  STIMULUS : process
  begin
    resetn <= '1';
    wait for (24*CLOCK_PERIOD);
    resetn <= '0';
    wait for (CLOCK_PERIOD);
  end process;

  COUNT : process
  begin
    count_en <= '1';
    wait for (15*CLOCK_PERIOD);
    count_en <= '0';
    wait for (4*CLOCK_PERIOD);
  end process;

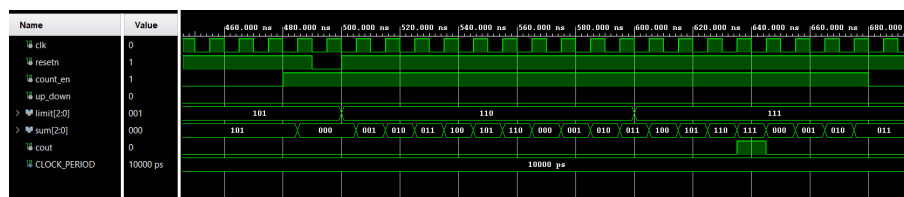
  GEN_LIMIT : process
  begin
    limit <= (limit) + 1;
    wait for (8*CLOCK_PERIOD);
  end process;

  GEN_CLK : process
  begin
    clk <= '0';
    wait for (CLOCK_PERIOD / 2);
    clk <= '1';
    wait for (CLOCK_PERIOD / 2);
  end process;

end tb;

```

Με την εκτέλεση της προσομοίωσης ενα αρκετά ενδεικτικό απόσπασμα για την ορθή λειτουργία του κώδικα είναι το παρακάτω:



Μπορούμε πράγματι να παρατηρήσουμε ότι ο μετρητής περιορίζεται κάθε φορά από το upper Limit που υπάρχει εκείνη την στιγμή

Θέμα 1: Ημιαθροιστής σε περιγραφή dataflow

Ο κώδικας σε vhdl για την περιγραφή του ημιαθροιστή και της αρχιτεκτονικής του σε dataflow vhdl είναι ο εξής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder is
  Port (
    i_a,i_b: in std_logic;

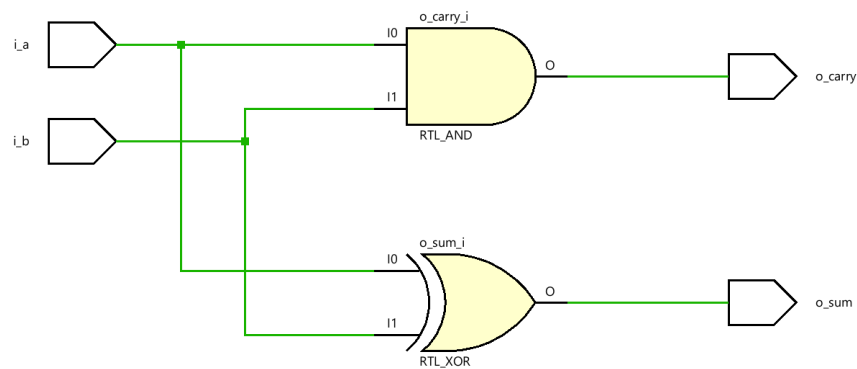
    o_sum ,o_carry : out std_logic
  );
end half_adder;

architecture Dataflow of half_adder is

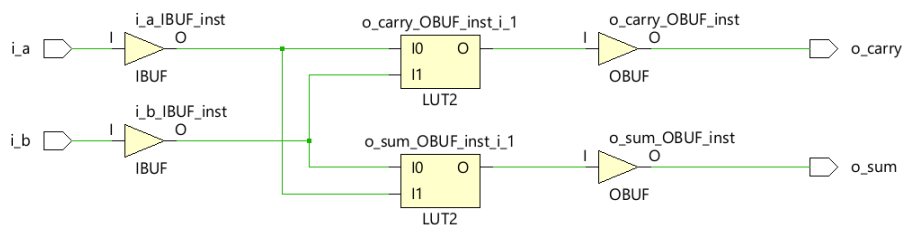
begin
  o_sum  <= i_a xor i_b;
  o_carry <= i_a and i_b;

end Dataflow;
```

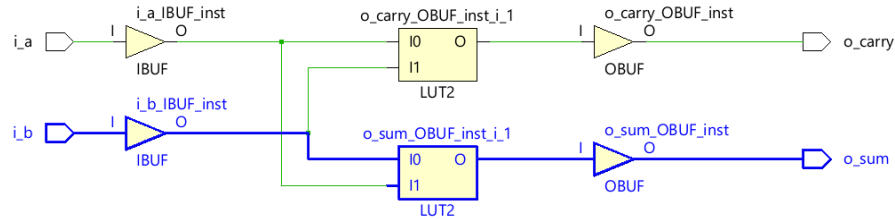
Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Με τη χρήση του synthesis παίρνουμε το εξής schematic:



Χρησιμοποιώντας το timing summary για το κύκλωμα μας μπορούμε να βρούμε το κρίσιμο μονοπάτι του κυκλώματος μας, το οποίο φαίνεται παρακάτω:



Όπως φαίνεται παρακάτω η χρονική καθυστέρηση του critical path του κυκλώματος είναι 5.377 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	∞	3	4	2	i_b	o_sum	5.377	3.778	1.599	∞	input port clock		
Path 2	∞	3	4	2	i_b	o_carry	5.351	3.752	1.599	∞	input port clock		

Ο εντοπισμός του κρίσιμου μονοπατιού είναι σημαντικός, καθώς μπορούμε να παρατηρήσουμε ποιο μονοπάτι δεδομένων είναι το πιο χρονοβόρο στην σχεδίαση μας. Έτσι, αν χρειαστούμε βελτιώσεις στον χρόνο ξέρουμε ποιο μονοπάτι χρειάζεται να αλλάξει. Επιπλέον, η μέγιστη χρονική καθυστέρηση επηρεάζει άμεσα την επιλογή της ελάχιστης συχνότητας ρολογιού.

Για την προσομοίωση και τον έλεγχο της σωστής λειτουργίας του κυκλώματός μας χρησιμοποιούμε το παρακάτω test-bench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder_tb is
-- Port ( );
end half_adder_tb;

architecture tb of half_adder_tb is

-----component-----
component half_adder is
    Port (
        i_a, i_b: in std_logic;

        o_sum ,o_carry : out std_logic
    );
end component;

-----signals-----
signal i_a,i_b,o_sum ,o_carry : std_logic := '0' ;

-----constant-----
constant CLOCK_PERIOD : time := 10 ns;

begin

    DUT: half_adder
    port map (
        i_a=>i_a,
        i_b=>i_b,

        o_sum=>o_sum,
        o_carry=>o_carry
    );
```

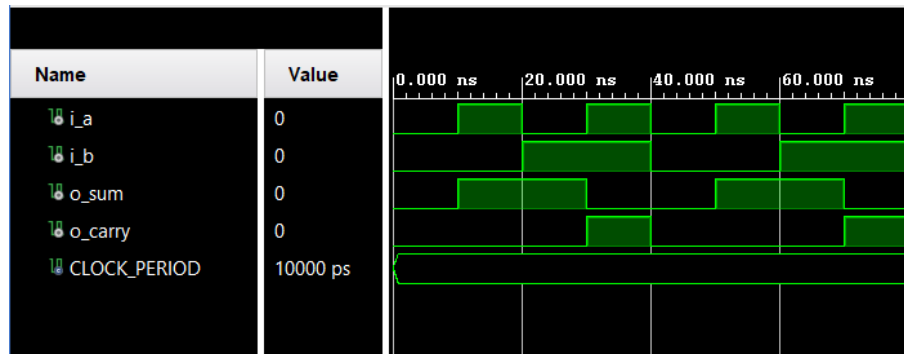
```

GEN_IA : process
begin
    i_a <= '0';
    wait for (CLOCK_PERIOD );
    i_a <= '1';
    wait for (CLOCK_PERIOD );
end process;

GEN_IB : process
begin
    i_b <= '0';
    wait for (2*CLOCK_PERIOD );
    i_b <= '1';
    wait for (2*CLOCK_PERIOD );
end process;

end tb;

```



Παρατηρούμε ότι έχουμε ορθά αποτελέσματα για όλους τους συνδυασμούς εισόδων.

Θέμα 2: Πλήρης αθροιστής σε περιγραφή structural

Ο κώδικας σε vhdl για την περιγραφή του αθροιστή και της αρχιτεκτονικής του σε structural vhdl είναι ο εξής:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    Port (
        i_a, i_b, i_cin : in std_logic;
        o_sum, o_carry : out std_logic
    );
end full_adder;

architecture structural of full_adder is

    component half_adder is
        Port (
            i_a, i_b : in std_logic;
            o_sum, o_carry : out std_logic
        );
    end component;

```

```

    );
end component;

signal o_sum_mid : std_logic;
signal o_carry_mid : std_logic;
signal o_c : std_logic;

begin

    HA1 : half_adder
    port map(
        i_a    => i_a,
        i_b    => i_b,
        o_sum   => o_sum_mid,
        o_carry => o_carry_mid
    );

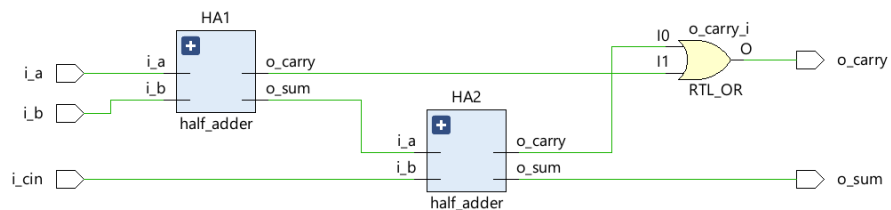
    HA2 : half_adder
    port map(
        i_a    => o_sum_mid,
        i_b    => i_cin,
        o_sum   => o_sum,
        o_carry => o_c
    );

    o_carry <= o_c or o_carry_mid;

end structural;

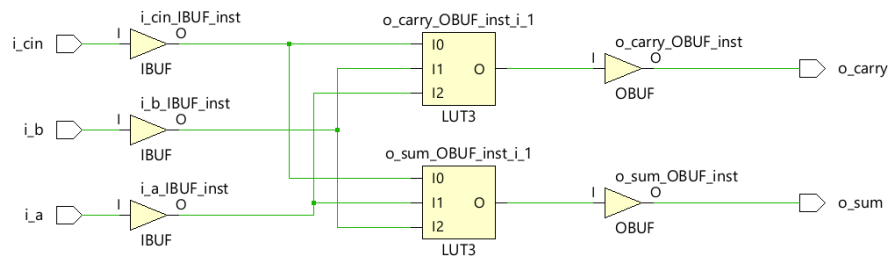
```

Με τη χρήση του elaborate design παίρνουμε το εξής schematic:

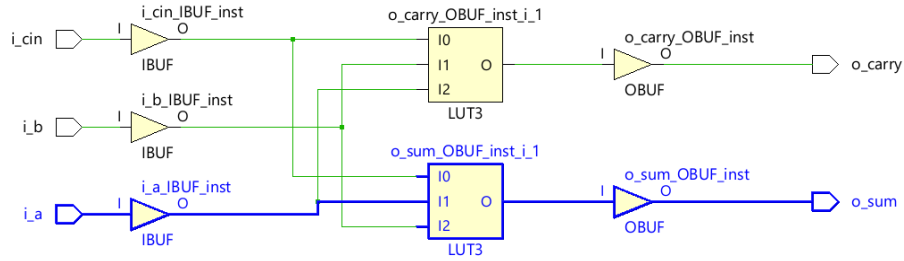


Μπορούμε να παρατηρήσουμε ότι όπως αναμένεται στο elaborated design ο πλήρης αθροιστής αποτελείται από 2 ημιαθροιστές και μία πύλη or.

Με τη χρήση του synthesis παίρνουμε το εξής schematic:



Χρησιμοποιώντας το timing summary για το κύκλωμα μας μπορούμε να βρούμε το κρίσιμο μονοπάτι του κυκλώματος μας, το οποίο φαίνεται παρακάτω:



Όπως φαίνεται παρακάτω η χρονική καθυστέρηση του critical path του κυκλώματος είναι 5.377 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception
Path 1	∞	3	4	2	i_a	o_sum	5.377	3.778	1.599	∞	input port clock		
Path 2	∞	3	4	2	i_a	o_carry	5.351	3.752	1.599	∞	input port clock		

Για την προσομοίωση και τον έλεγχο της σωστής λειτουργίας του κυκλώματός μας χρησιμοποιούμε το παρακάτω test-bench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder_tb is
-- Port ( );
end full_adder_tb;

architecture tb of full_adder_tb is

    component full_adder is
        Port (
            i_a, i_b, i_cin      : in std_logic;
            o_sum, o_carry      : out std_logic
        );
    end component;

    signal i_a, i_b, i_cin, o_sum, o_carry : std_logic := '0' ;

    constant CLOCK_PERIOD : time := 10 ns;

begin

    DUT : full_adder
        port map
        (
            i_a => i_a,
            i_b => i_b,
            i_cin => i_cin,
            o_sum => o_sum,
            o_carry => o_carry
        );

    IN_A : process
    begin
        i_a <= '0';
        wait for (CLOCK_PERIOD );
        i_a <= '1';
    end process;
end;
```

```

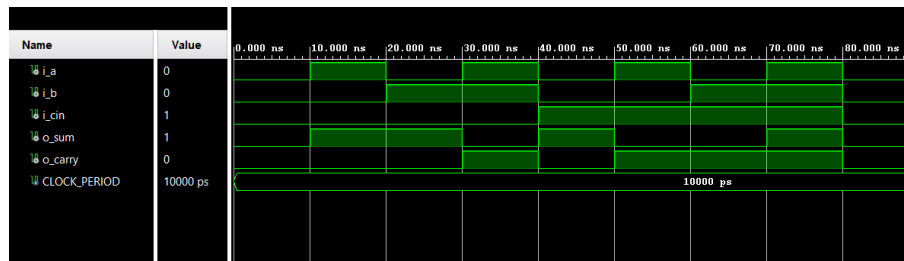
        wait for (CLOCK_PERIOD );
    end process;

    IN_B : process
    begin
        i_b <= '0';
        wait for (2*CLOCK_PERIOD );
        i_b <= '1';
        wait for (2*CLOCK_PERIOD );
    end process;

    IN_carry : process
    begin
        i_cin <= '0';
        wait for (4*CLOCK_PERIOD );
        i_cin <= '1';
        wait for (4*CLOCK_PERIOD );
    end process;

end tb;

```



Παρατηρούμε ότι έχουμε ορθά αποτελέσματα για όλους τους συνδυασμούς εισόδων.

Θέμα 3: Παράλληλος αθροιστής των 4 bit σε περιγραφή structural

Ο κώδικας σε vhdl για την περιγραφή του παράλληλου αθροιστή και της αρχιτεκτονικής του σε structural vhdl είναι ο εξής:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity FA_4bit is
    Port (
        i_a ,i_b :in std_logic_vector (3 downto 0);
        i_cin : in std_logic;
        o_sum :out std_logic_vector (3 downto 0);
        o_cout : out std_logic
    );
end FA_4bit;

architecture structural of FA_4bit is

    component full_adder is
        Port (
            i_a, i_b,i_cin      : in std_logic;

```

```

        o_sum, o_carry : out std_logic
    );
end component;

signal carry01, carry12, carry23 : std_logic := '0';

begin

    FA1 : full_adder
        port map (
            i_a      => i_a(0),
            i_b      => i_b(0),
            i_cin    => i_cin,
            o_sum    => o_sum(0),
            o_carry  => carry01
        );

    FA2 : full_adder
        port map (
            i_a      => i_a(1),
            i_b      => i_b(1),
            i_cin    => carry01,
            o_sum    => o_sum(1),
            o_carry  => carry12
        );

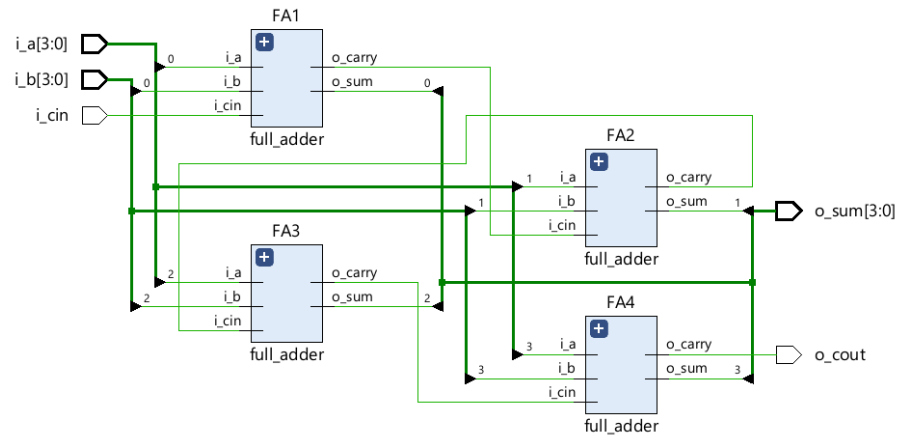
    FA3 : full_adder
        port map (
            i_a      => i_a(2),
            i_b      => i_b(2),
            i_cin    => carry12,
            o_sum    => o_sum(2),
            o_carry  => carry23
        );

    FA4 : full_adder
        port map (
            i_a      => i_a(3),
            i_b      => i_b(3),
            i_cin    => carry23,
            o_sum    => o_sum(3),
            o_carry  => o_cout
        );

end structural;

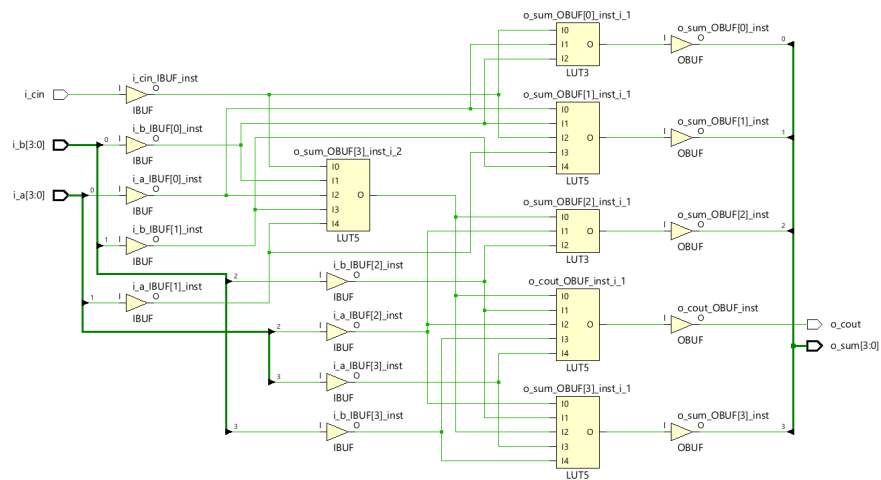
```

Με τη χρήση του elaborate design παίρνουμε το εξής schematic:

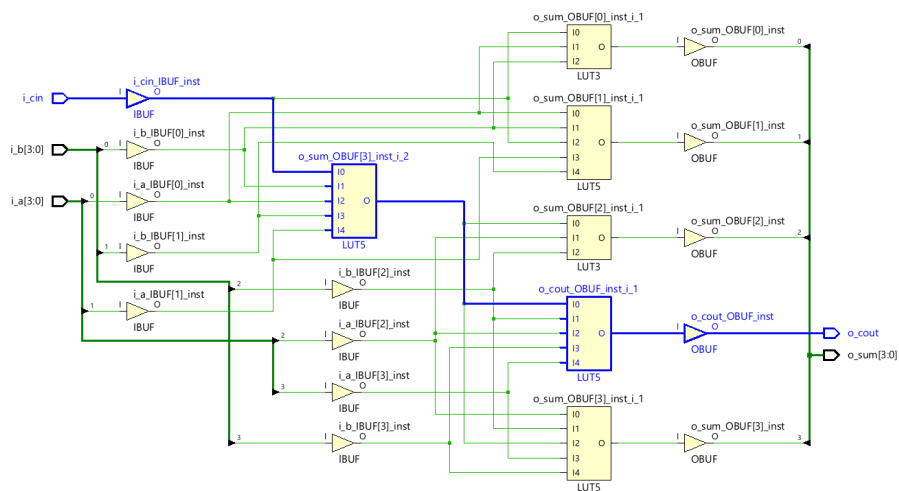


Μπορούμε να παρατηρήσουμε ότι όπως αναμένεται στο elaborated design ο παράλληλος αθροιστής αποτελείται από 4 πλήρεις αθροιστές.

Με τη χρήση του synthesis παίρνουμε το εξής schematic:



Χρησιμοποιώντας το timing summary για το κύκλωμα μας μπορούμε να βρούμε το κρίσιμο μονοπάτι του κυκλώματος μας, το οποίο φαίνεται παρακάτω:



Όπως φαίνεται παρακάτω η χρονική καθυστέρηση του critical path του κυκλώματος είναι 5.970 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exce
Path 1	∞	4	5	3	i_cin	o_cout	5.970	3.904	2.066	∞	input port clock		
Path 2	∞	4	5	3	i_b[0]	o_cout	5.970	3.904	2.066	∞	input port clock		
Path 3	∞	4	5	3	i_cin	o_cout	5.970	3.904	2.066	∞	input port clock		
Path 4	∞	4	5	3	i_b[0]	o_cout	5.970	3.904	2.066	∞	input port clock		
Path 5	∞	4	5	3	i_cin	o_sum[2]	5.970	3.904	2.066	∞	input port clock		
Path 6	∞	4	5	3	i_b[0]	o_sum[2]	5.970	3.904	2.066	∞	input port clock		
Path 7	∞	4	5	3	i_cin	o_sum[2]	5.970	3.904	2.066	∞	input port clock		

Για την προσομοίωση και τον έλεγχο της σωστής λειτουργίας του κυκλώματός μας χρησιμοποιούμε το παρακάτω test-bench:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity FA_4bit_tb is
-- Port ( );
end FA_4bit_tb;

architecture tb of FA_4bit_tb is

component FA_4bit is
Port (
    i_a ,i_b :in std_logic_vector (3 downto 0);
    i_cin : in std_logic;
    o_sum :out std_logic_vector (3 downto 0);
    o_cout : out std_logic
);
end component;

signal i_cin ,o_cout :std_logic := '0';
signal i_a, i_b , o_sum :std_logic_vector(3 downto 0) := "0000";

constant CLOCK_PERIOD : time := 10 ns;

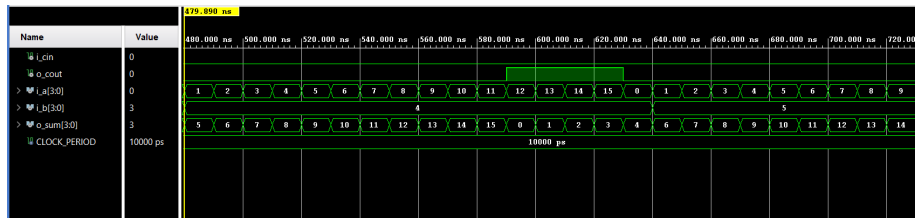
begin

    DUT : FA_4bit
    port map(
        i_a=>i_a,
        i_b=>i_b,
        o_sum=>o_sum,
        i_cin=>i_cin,
        o_cout=>o_cout
    );

    IN_A : process
    begin
        i_a <= i_a+1;
        wait for (CLOCK_PERIOD );
    end process;

    IN_B : process
    begin
        i_b <= i_b+1;
        wait for (16*CLOCK_PERIOD);
    end process;
```

```
end tb;
```



Παρατηρούμε ότι έχουμε ορθά αποτελέσματα για όλους τους συνδυασμούς εισόδων που δίνονται. Σημειώνεται ότι δεν είναι όλοι οι πιθανοί συνδυασμοί εισόδων γιατί είναι πάρα πολλοί, αλλά είναι αντιπροσωπευτικοί συνδυασμοί για την επαλήθευση της ορθής λειτουργίας του κυκλώματος.

Θέμα 3: BCD Full Adder σε περιγραφή structural

Στην αναπαράσταση BCD (Binary Coded Decimal) κάθε δεκαδικό ψηφίο ενός αριθμού αναπαρίσταται ξεχωριστά σε δυαδική μορφή. Για παράδειγμα, ο αριθμός 12 σε BCD γράφεται ως 0001 0010, όπου το 0001 αντιστοιχεί στο ψηφίο 1 και το 0010 στο ψηφίο 2. Για τη λειτουργία του BCD αθροιστή ισχύει ότι όταν το αποτέλεσμα της πρόσθεσης είναι μικρότερο ή ίσο του 9, τότε το cout είναι 0. Αντίθετα, όταν το αποτέλεσμα της πρόσθεσης είναι μεγαλύτερο από 9, τότε το cout του πρώτου adder γίνεται 1. Το σήμα αυτό χρησιμοποιείται στη συνέχεια ώστε να προστεθεί στο ενδιάμεσο άθροισμα που προκύπτει από τον πρώτο adder ο αριθμός 6, για να οδηγήσουμε πάντα στο σωστό BCD αποτέλεσμα.

Ο κώδικας σε vhdl για την περιγραφή του bcd full adder και της αρχιτεκτονικής του σε structural vhdl είναι ο εξής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BDC_adder is
    Port (
        i_a ,i_b :in std_logic_vector (3 downto 0);
        i_cin : in std_logic;
        o_bcd0 :out std_logic_vector (3 downto 0);
        o_bcd1: out std_logic
    );
end BDC_adder;

architecture Structural of BDC_adder is

    component FA_4bit is
        Port (
            i_a ,i_b :in std_logic_vector (3 downto 0);
            i_cin : in std_logic;
            o_sum :out std_logic_vector (3 downto 0);
            o_cout : out std_logic
        );
    end component;

    signal o_cout :std_logic := '0';
    signal o_sum :std_logic_vector(3 downto 0) := "0000";
    signal checker: std_logic :='0';
begin
    FA1 : FA_4bit
    port map(
        i_a=>i_a,
        i_b=>i_b,
```

```

    i_cin=>i_cin,
    o_sum=>o_sum,
    o_cout=>o_cout
);

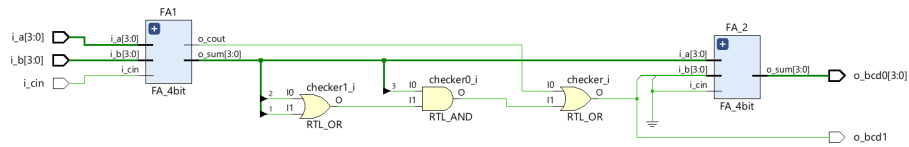
checker<=o_cout or ((o_sum(3)) and (o_sum(2) or o_sum(1)));
o_bcd1<=checker;

FA_2: FA_4bit
port map(
    i_a=>o_sum,
    i_b=>'0'&checker&checker&'0',
    i_cin=>'0',
    o_sum=>o_bcd0
);

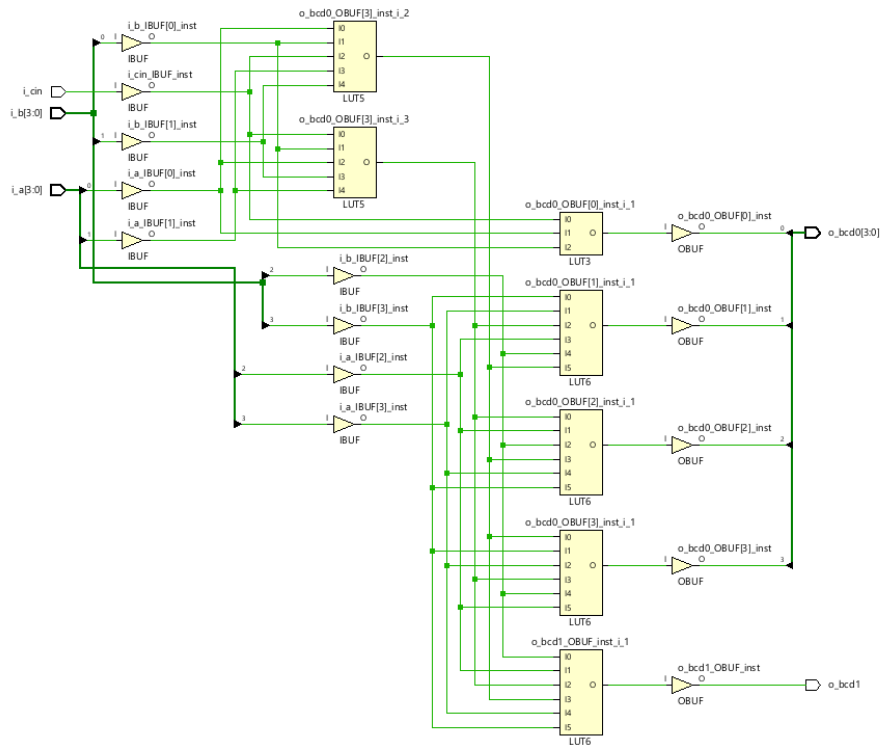
end Structural;

```

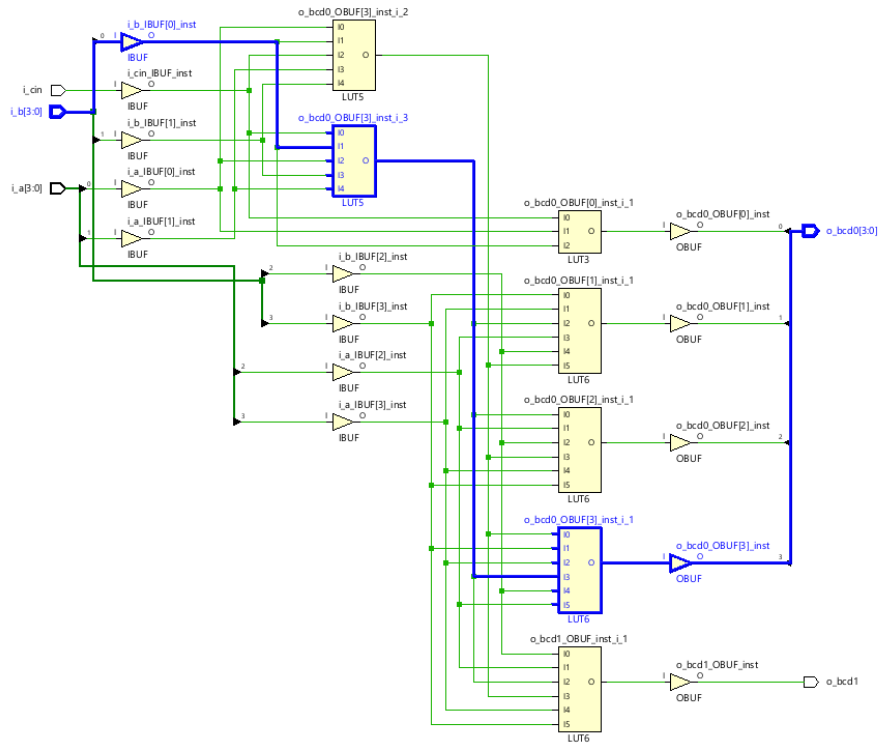
Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Με τη χρήση του synthesis παίρνουμε το εξής schematic:



Χρησιμοποιώντας το timing summary για το κύκλωμα μας μπορούμε να βρούμε το κρίσιμο μονοπάτι του κυκλώματος μας, το οποίο φαίνεται παρακάτω:



Όπως φαίνεται παρακάτω η χρονική καθυστέρηση του critical path του κυκλώματος είναι 5.976 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Excepti
Path 1	∞	4	5	4	i_b[0]	o_bcd0[3]	5.976	3.904	2.072	∞	input port clock		
Path 2	∞	4	5	4	i_b[1]	o_bcd0[1]	5.948	3.876	2.072	∞	input port clock		
Path 3	∞	4	5	4	i_b[1]	o_bcd0[2]	5.948	3.876	2.072	∞	input port clock		

Για την προσομίωση και τον έλεγχο της σωστής λειτουργίας του κυκλώματός μας χρησιμοποιούμε το παρακάτω test-bench:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity bcd_adder_tb is
-- Port ( );
end bcd_adder_tb;

architecture tb of bcd_adder_tb is

component BDC_adder is
Port (
i_a ,i_b :in std_logic_vector (3 downto 0);
i_cin : in std_logic;
o_bcd0 :out std_logic_vector (3 downto 0);
o_bcd1: out std_logic
);
end component;

signal i_a,i_b,o_bcd0 : std_logic_vector (3 downto 0) := "0000";
signal i_cin ,o_bcd1 : std_logic := '0';

constant CLOCK_PERIOD : time := 10 ns;

```

```

begin
    DUT: BDC_adder
    port map (

        i_a=>i_a,
        i_b=>i_b,
        i_cin=>i_cin,
        o_bcd0=>o_bcd0,
        o_bcd1=>o_bcd1

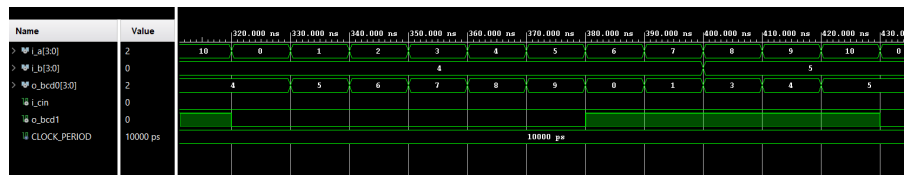
    );

    IN_A : process
    begin
        if i_a = "1010" then
            i_a <= "0000";
        else
            i_a <= i_a + 1;
        end if;
        wait for CLOCK_PERIOD;
    end process;

    IN_B : process
    begin
        if i_b = "1010" then
            i_b <= "0000";
        else
            i_b <= i_b + 1;
        end if;
        wait for 10 * CLOCK_PERIOD;
    end process;

end tb;

```



```

entity BCD_4digits is
  Port(
    i_a0, i_a1, i_a2, i_a3 : in std_logic_vector(3 downto 0);
    i_b0, i_b1, i_b2, i_b3 : in std_logic_vector(3 downto 0);
    o_bcd0, o_bcd1, o_bcd2, o_bcd3 : out std_logic_vector(3 downto 0);
    o_bcd_final : out std_logic
  );
end BCD_4digits;

architecture structural of BCD_4digits is

  component BDC_adder is
    Port (
      i_a, i_b : in std_logic_vector(3 downto 0);
      i_cin : in std_logic;
      o_bcd0 : out std_logic_vector(3 downto 0);
      o_bcd1 : out std_logic
    );
  end component;

  signal bcd01, bcd12, bcd23 : std_logic := '0';

begin

  BCD0 : BDC_adder
    port map(
      i_a => i_a0,
      i_b => i_b0,
      i_cin => '0',
      o_bcd0 => o_bcd0,
      o_bcd1 => bcd01
    );

  BCD1 : BDC_adder
    port map(
      i_a => i_a1,
      i_b => i_b1,
      i_cin => bcd01,
      o_bcd0 => o_bcd1,
      o_bcd1 => bcd12
    );

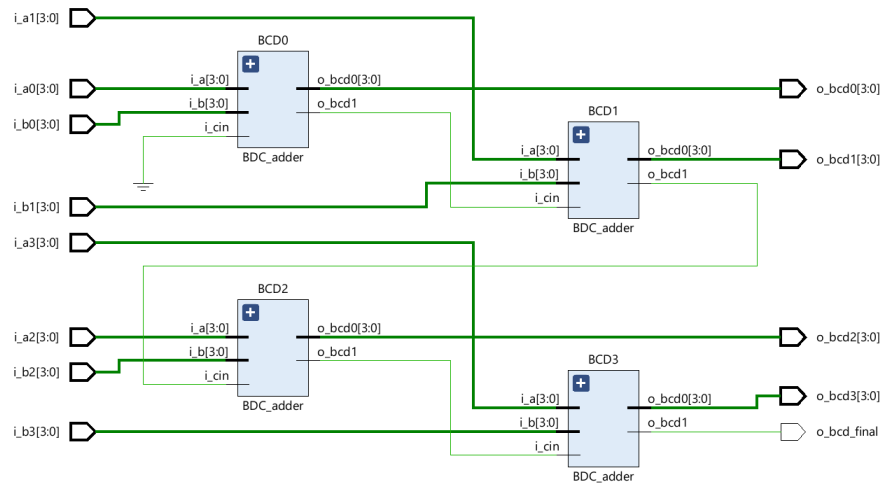
  BCD2 : BDC_adder
    port map(
      i_a => i_a2,
      i_b => i_b2,
      i_cin => bcd12,
      o_bcd0 => o_bcd2,
      o_bcd1 => bcd23
    );

  BCD3 : BDC_adder
    port map(
      i_a => i_a3,
      i_b => i_b3,
      i_cin => bcd23,
      o_bcd0 => o_bcd3,
      o_bcd1 => o_bcd_final
    );

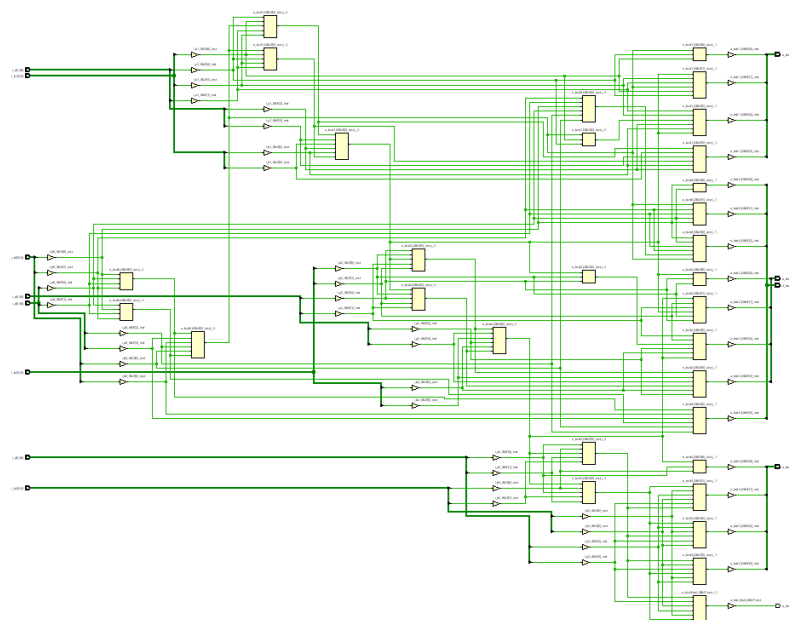
```

```
end structural;
```

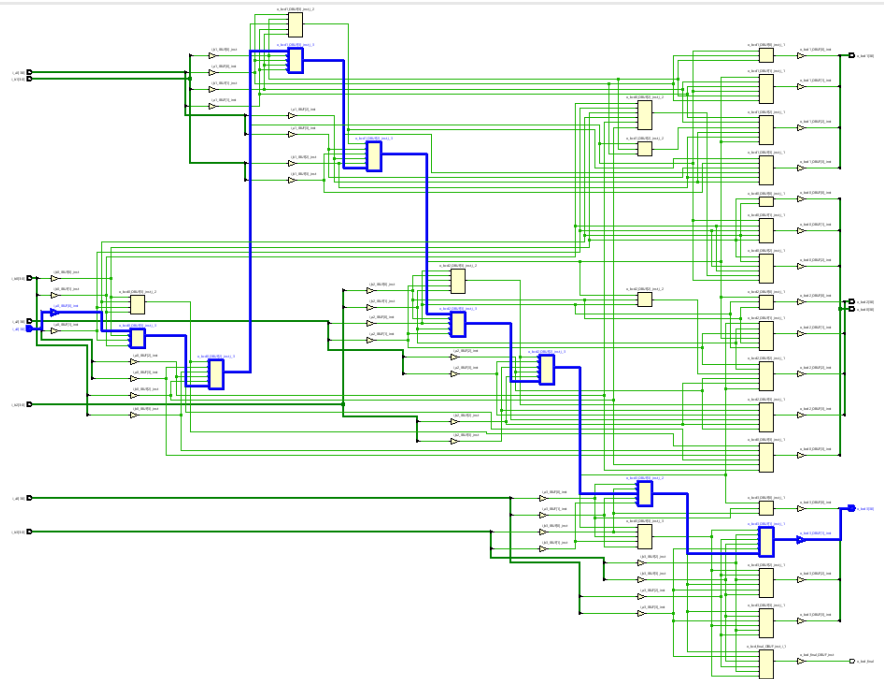
Με τη χρήση του elaborate design παίρνουμε το εξής schematic:



Με τη χρήση του synthesis παίρνουμε το εξής schematic:



Χρησιμοποιώντας το timing summary για το κύκλωμα μας μπορούμε να βρούμε το κρίσιμο μονοπάτι του κυκλώματος μας, το οποίο φαίνεται παρακάτω:



Όπως φαίνεται παρακάτω η χρονική καθυστέρηση του critical path του κυκλώματος είναι 9.533 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 1	∞	10	11	7	i_a0[0]	o_bcd3[1]	9.533	4.636	4.897	∞	input port clock	
Path 2	∞	10	11	7	i_a0[0]	o_bcd3[2]	9.533	4.636	4.897	∞	input port clock	
Path 3	∞	10	11	7	i_a0[0]	o_bcd3[3]	9.527	4.630	4.897	∞	input port clock	
Path 4	∞	10	11	7	i_a0[0]	o_bcd_final	9.527	4.630	4.897	∞	input port clock	
Path 5	∞	9	10	7	i_a0[0]	o_bcd2[1]	8.936	4.512	4.424	∞	input port clock	

Για την προσομοίωση και τον έλεγχο της σωστής λειτουργίας του κυκλώματός μας χρησιμοποιούμε το παρακάτω test-bench:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity BCD_4dig_tb is
-- Port ( );
end BCD_4dig_tb;

architecture tb of BCD_4dig_tb is

component BCD_4digits is
Port(
    i_a0, i_a1, i_a2, i_a3 : in std_logic_vector(3 downto 0);
    i_b0, i_b1, i_b2, i_b3 : in std_logic_vector(3 downto 0);
    o_bcd0, o_bcd1, o_bcd2, o_bcd3 : out std_logic_vector(3 downto 0);
    o_bcd_final : out std_logic
);
end component;

signal i_a0, i_a1, i_a2, i_a3 : std_logic_vector(3 downto 0):=(others => '0');
signal i_b0, i_b1, i_b2, i_b3 : std_logic_vector(3 downto 0):=(others => '0');
signal o_bcd0,o_bcd1,o_bcd2,o_bcd3: std_logic_vector(3 downto 0):= (others => '0');
signal o_bcd_final :std_logic:='0';
constant CLOCK_PERIOD : time := 10 ns;

```

```

begin

DUT : BCD_4digits
  port map(
    i_a0      => i_a0,
    i_a1      => i_a1,
    i_a2      => i_a2,
    i_a3      => i_a3,
    i_b0      => i_b0,
    i_b1      => i_b1,
    i_b2      => i_b2,
    i_b3      => i_b3,
    o_bcd0     => o_bcd0,
    o_bcd1     => o_bcd1,
    o_bcd2     => o_bcd2,
    o_bcd3     => o_bcd3,
    o_bcd_final => o_bcd_final
  );

stimulus : process
begin

  -- TEST 1: 0000 + 0000 = 0000
  i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "0000";
  i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0000";
  wait for 2*CLOCK_PERIOD ;

  -- TEST 2: 0001 + 0002 = 0003
  i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "0001";
  i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0010";
  wait for 2*CLOCK_PERIOD ;

  -- TEST 3: 0005 + 0004 = 0009
  i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "0101";
  i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0100";
  wait for 2*CLOCK_PERIOD ;

  -- TEST 4: 0005 + 0005 = 0010
  i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "0101";
  i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0101";
  wait for 2*CLOCK_PERIOD ;

  -- TEST 5: 0009 + 0001 = 0010
  i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "0000"; i_a0 <= "1001";
  i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0001";
  wait for 2*CLOCK_PERIOD ;

  -- TEST 6: 0099 + 0001 = 0100
  i_a3 <= "0000"; i_a2 <= "0000"; i_a1 <= "1001"; i_a0 <= "1001";
  i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0001";
  wait for 2*CLOCK_PERIOD ;

  -- TEST 7: 0999 + 0001 = 1000
  i_a3 <= "0000"; i_a2 <= "1001"; i_a1 <= "1001"; i_a0 <= "1001";
  i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0001";
  wait for 2*CLOCK_PERIOD ;

  -- TEST 8: 1234 + 4321 = 5555
  i_a3 <= "0001"; i_a2 <= "0010"; i_a1 <= "0011"; i_a0 <= "0100";

```

```

i_b3 <= "0100"; i_b2 <= "0011"; i_b1 <= "0010"; i_b0 <= "0001";
wait for 2*CLOCK_PERIOD ;

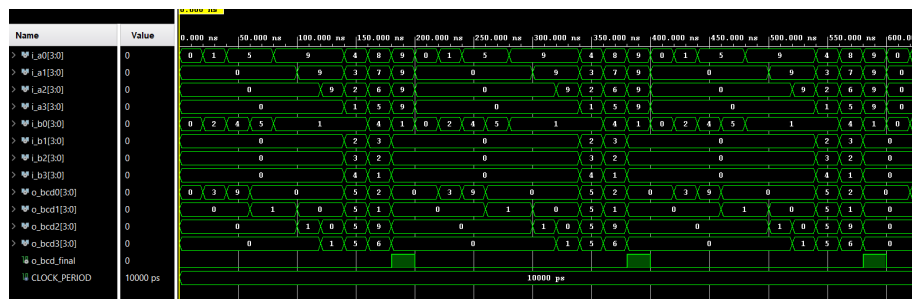
-- TEST 9: 5678 + 1234 = 6912
i_a3 <= "0101"; i_a2 <= "0110"; i_a1 <= "0111"; i_a0 <= "1000";
i_b3 <= "0001"; i_b2 <= "0010"; i_b1 <= "0011"; i_b0 <= "0100";
wait for 2*CLOCK_PERIOD ;

-- TEST 10: 9999 + 0001 = 10000
i_a3 <= "1001"; i_a2 <= "1001"; i_a1 <= "1001"; i_a0 <= "1001";
i_b3 <= "0000"; i_b2 <= "0000"; i_b1 <= "0000"; i_b0 <= "0001";
wait for 2*CLOCK_PERIOD ;

end process;

end tb;

```



Παρατηρούμε ότι έχουμε ορθά αποτελέσματα για όλους τους συνδυασμούς εισόδων που δίνονται. Σημειώνεται ότι δεν είναι όλοι οι πιθανοί συνδυασμοί εισόδων γιατί είναι πάρα πολλοί , αλλά είναι αντιπροσωπευτικοί συνδυασμοί για την επαλήθευση της ορθής λειτουργίας του κυκλώματος..